Kettering University
College of Engineering
Department of Electrical and Computer Engineering

Final Project Report

Course No. & Title: EE-490 - Senior EE Design Project

Instructor's Name: Dr. Chen Duan

Title: Final Project Report

Date Submitted: September 20, 2026

Team Members:

(1) Michael Collins

(2) Matthew Herber

(3) Dillon Peterson

(4) Ryan Rabideau

INSTRUCTOR SECTION

Comments: _____

_____

_____

_____

TABLE OF CONTENTS

Section **Page**

TITLE PAGE ……..1

TABLE OF CONTENTS ……....2

ACKNOWLEDGEMENTS ……...…….4

1. INTRODUCTION AND OVERVIEW ……...…...….5

2. OVERALL BMS SYSTEM DESIGN …….…...…...….6

3. BATTERY PACK CONFIGURATION ……..…….8

4. MICROCONTROLLER CONFIGURATION……...……...……9

5. ANALOG FRONT END CONFIGURATION ……...……11

6. BATTERY MEASUREMENTS ……..12

6.1 Cell Voltages....……...……...……....……12

6.2 Pack Current..……...……...…….……..……12

6.3 Cell Temperatures...……...…….……13

7. FAULT PROTECTIONS ……...……..14

7.1 Over and Under Cell Voltage……...……...14

7.2 Over Pack Current...…….……...……15

7.3 Over Cell Temperature...…….……...……15

8. ACTIVE BALANCING ……...……...…….16

9. DATA TELEMETRY AND PC INTERFACE……...……...…….18

9.1 UART Data Telemetry……...…….18

9.2 LabVIEW PC Interface……...18

10. SOLDERING AND BUILD……...……...…….20

11. ENCLOSURE ……...……...……...…….22

12. BILL OF MATERIALS AND BUDGET……...…….27

13. PROJECT MANAGEMENT……..…….…….29

14. WORKLOAD DISTRIBUTION……..……...…….30

14.1 Michael Collins ….……...…..30

14.2 Matthew Herber ……...……..30

14.3 Dillon Peterson ……..……...……..30

14.4 Ryan Rabideau …….……..30

14.5 Overall Workload Estimate ……..……..31

15. FINAL PROJECT OUTCOMES……..…...…….32

16. CONCLUSIONS ……...……..…….34

ACKNOWLEDGEMENTS

We would like to thank Dr. Chen Duan, Assistant Professor of Electrical Engineering, for his instruction and direction during the time of this project and course.

We would also like to thank Mr. Scott LaForest for all procurement of the many critical parts and materials used in the project.

1. INTRODUCTION AND OVERVIEW

Lithium-ion batteries are commonly used in portable electronics, electric vehicles, and energy storage systems because of their high energy density and rechargeable design. When multiple lithium-ion cells are connected to form a battery pack, a battery management system (BMS) is needed to monitor the individual cells and protect the pack from unsafe operating conditions. A BMS can also help maintain balanced cell voltages during charging and discharging.

The objective of this senior design project was to design and build an active battery management system for a nominal 24 V lithium-ion battery pack. The battery pack uses a 2P7S configuration, consisting of seven series-connected cell groups with two lithium-ion cells connected in parallel within each group. This configuration provides a nominal pack voltage of approximately 25.2 V and a maximum voltage of 29.4 V when fully charged.

The system uses an STM32F411RE microcontroller and an ST L9963E battery-monitoring integrated circuit to monitor battery conditions and control the system. It measures individual cell-group voltages, pack current, and battery temperatures. These measurements are used to evaluate protection conditions, control the master relay, and provide operating information to the user.

The project also includes an active-balancing system designed to reduce voltage differences between the series-connected cell groups. A LabVIEW PC interface displays battery measurements, allowing the user to monitor the system during operation. The battery pack, electronics, and supporting components are housed in a physical assembly designed and constructed by the team.

The project required the integration of battery-pack construction, electrical circuit design, embedded firmware development, mechanical design, and system testing. The following sections describe the design and implementation of the individual subsystems, the testing and integration process, and the final project outcomes.

2. OVERALL BMS SYSTEM DESIGN

The battery management system was designed around a nominal 24 V lithium-ion battery pack using a 2P7S configuration. The system consists of the battery pack, battery-monitoring hardware, an STM32F411RE microcontroller, current and temperature sensing circuitry, active-balancing hardware, a master relay, and a LabVIEW PC interface. These components work together to monitor the battery pack, control its operation, and provide information to the user.

The battery pack contains seven series-connected cell groups, with two lithium-ion cells connected in parallel within each group. The ST L9963E analog front end (AFE) monitors the individual cell-group voltages and communicates the measurements to the STM32 microcontroller. Additional sensing circuitry provides pack-current and battery-temperature measurements.

The STM32F411RE serves as the main controller for the system. During operation, the firmware collects the battery measurements and evaluates the implemented protection conditions. These include cell overvoltage, cell undervoltage, pack overcurrent, and battery overtemperature. The microcontroller uses a digital output to control the master relay, which allows the battery pack to be disconnected from the external circuit when an unsafe operating condition is detected.

The system also includes active-balancing circuitry to help reduce voltage differences between the series-connected cell groups. The microcontroller controls the balancing switches through a repeating switching sequence. The design and operation of the active-balancing circuit are discussed in Section 8.

Battery measurements and system-status information are transmitted from the STM32 to the computer through UART communication. A LabVIEW application receives the data and displays the individual cell-group voltages, total pack voltage, battery temperatures, and pack current. This allows the user to observe the battery's operating conditions during testing and operation.

The battery pack, monitoring and control electronics, and supporting hardware were assembled into the final prototype. The physical construction and enclosure design are discussed in Sections 10 and 11.

Overall, the system combines battery monitoring, protection, active balancing, and PC communication into a single prototype. Each subsystem performs a specific function, while the STM32 coordinates the measurement processing, protection decisions, relay control, and communication required for operation.

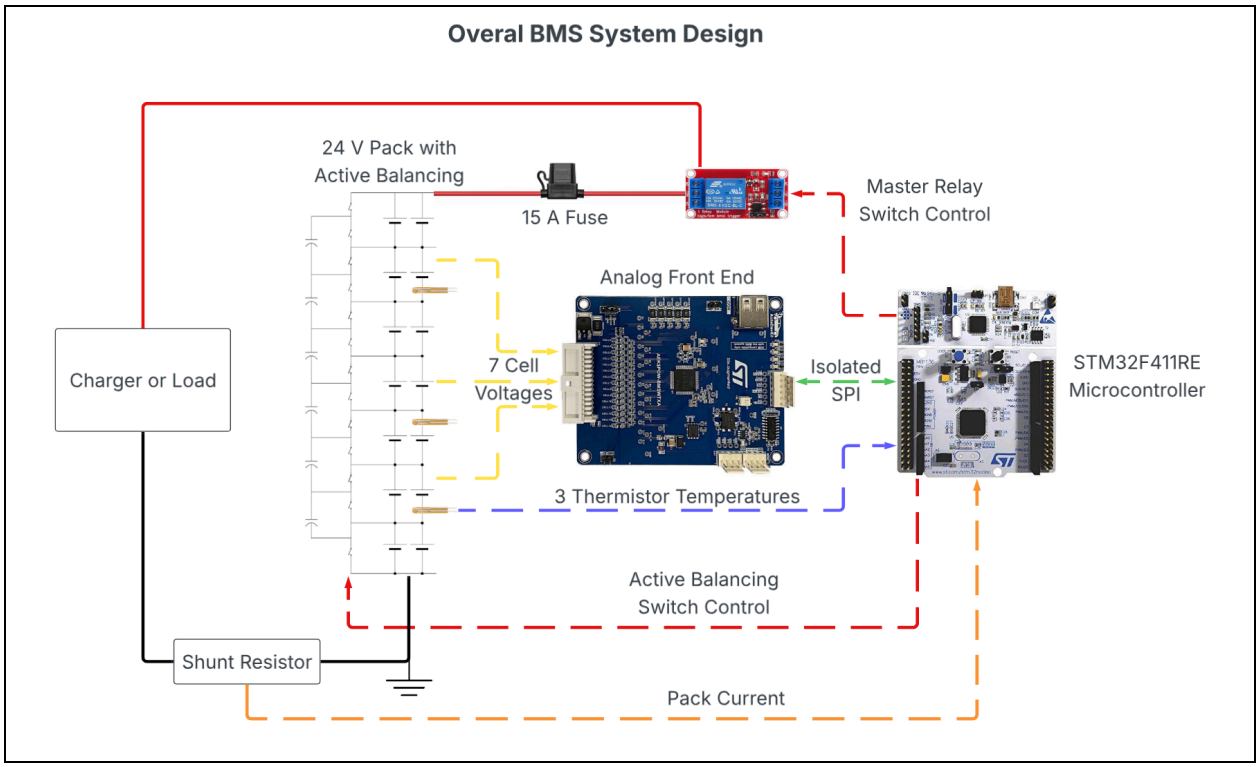


Figure 1. Overall BMS System Design

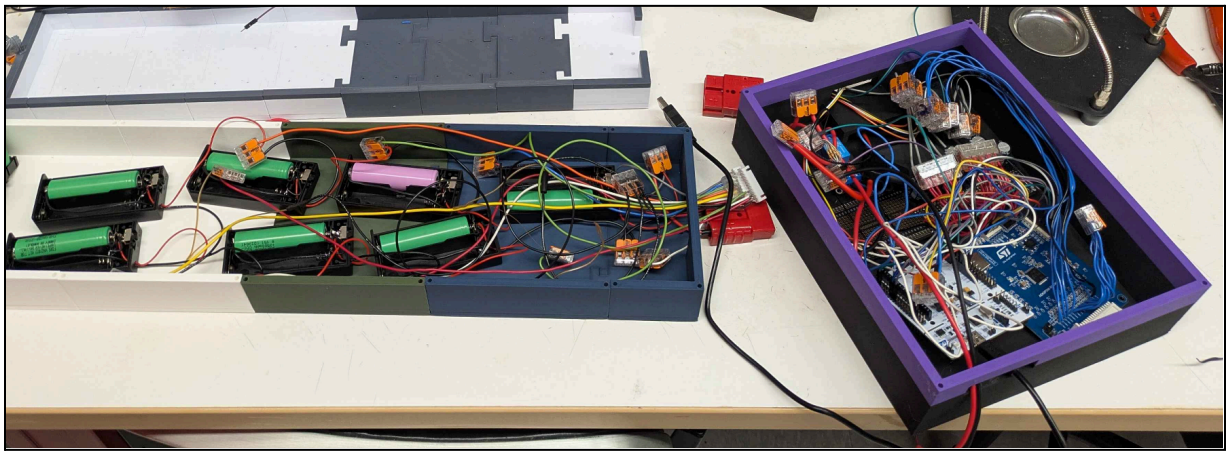


Figure 2. Final BMS Prototype System Build

3. BATTERY PACK CONFIGURATION

The battery pack design had a two-in-parallel and seven-in-series configuration (2P7S). This means that each module consists of two cells in parallel with one another in order to increase the current rating of the module. Seven of these parallel modules were then put into series with one another in order to build the desired voltage level required for the pack of 24 V. Seven cells will give a nominal voltage between about 23 and 29 V, depending on their SOC, so a buck-boost converter was used to regulate the pack to the desired 24 V output. Seven cells in series were required in order to meet this 24 V requirement, but putting two cells in parallel in each module was used in order to better understand how it would increase current and energy capacities of the pack. Despite the design being able to implement a full 14-cell 2P7S configuration, for many tests and demonstrations, only a 1P7S configuration was used, as it is easier and safer to debug and utilize. Otherwise, there was no limitation for adding a second cell to each module.

4. MICROCONTROLLER CONFIGURATION

The battery management system uses an STM32F411RE microcontroller on a NUCLEO-F411RE development board. The microcontroller serves as the main controller for the system by collecting battery measurements, evaluating protection conditions, controlling the master relay, and transmitting battery information to the PC interface.

The firmware was developed in C using STM32CubeIDE and the STM32 Hardware Abstraction Layer (HAL). STM32CubeMX was used to configure the microcontroller's peripherals and GPIO pins. The configuration includes SPI communication with the battery-monitoring hardware, analog-to-digital conversion for additional sensor measurements, UART communication with the PC, and digital outputs for relay and balancing control.

The STM32 communicates with the battery-monitoring hardware through SPI2. The L9963E battery-monitoring IC provides individual cell-voltage measurements, which are processed by the microcontroller and used for monitoring and protection. The microcontroller also receives temperature and pack-current measurements through the system's sensing circuitry.

The master relay is controlled by a dedicated GPIO output named MASTER_SWITCH. The firmware enables this output only when the monitored battery conditions satisfy the implemented protection limits. If an unsafe condition is detected, the firmware disables the output to disconnect the battery through the relay-control circuit.

The final firmware uses a continuously repeating main loop or “superloop” rather than a separate real-time operating system task for each function. During each loop, the microcontroller collects battery measurements, checks the protection conditions, updates the master relay, sends measurement data to the PC interface, and performs the active-balancing switching sequence. This approach allows the system's main functions to operate within a single application loop. The software flowchart for the microcontroller’s operation can be seen in Figure 3 below.

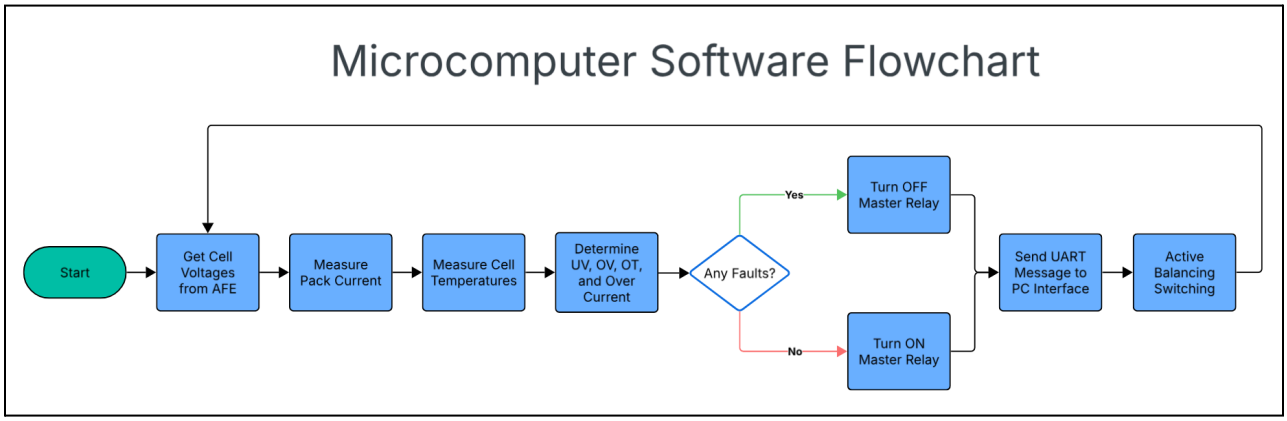


Figure 3. STM32F411RE microcontroller software flowchart of “superloop” application.

The STM32CubeMX pin configuration used for the final system is shown in Figure 4.

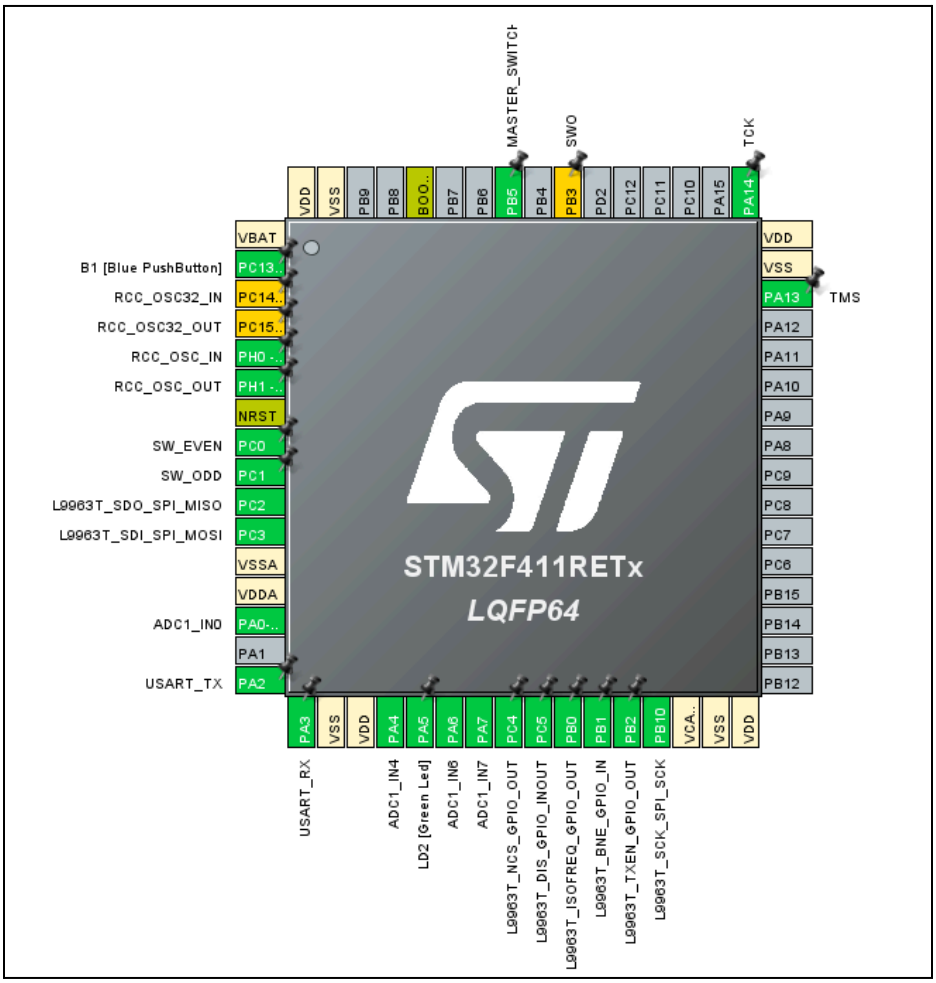


Figure 4. STM32F411RE microcontroller pin configuration in STM32CubeMX.

5. ANALOG FRONT END CONFIGURATION

The analog front end (AFE) used was a L9963E chip, which is produced by STMicroelectronics. This chip was used with the evaluation board AEK-POW-BMSWTX, which is also manufactured by STMicroelectronics. The primary purpose of the AFE was to constantly obtain the seven cell voltages and report these readings back to the microcontroller. This allows for the voltage connections from the battery pack to be isolated from the microcontroller. While this AFE chip could provide many other BMS functions, such as passive balancing and GPIO, only the cell voltage reading functionality was used.

The microcontroller was connected and talked to the AFE through an isolated SPI protocol. The most difficult part of using the AFE was ensuring a proper communication handshake between it and the microcontroller. Additionally, each register of the AFE had to be properly configured, and proper SPI message frames had to be sent in order to command voltage readings to be taken as well as read back to the microcontroller. An open source and use library was used in order to better try and communicate with the AFE.

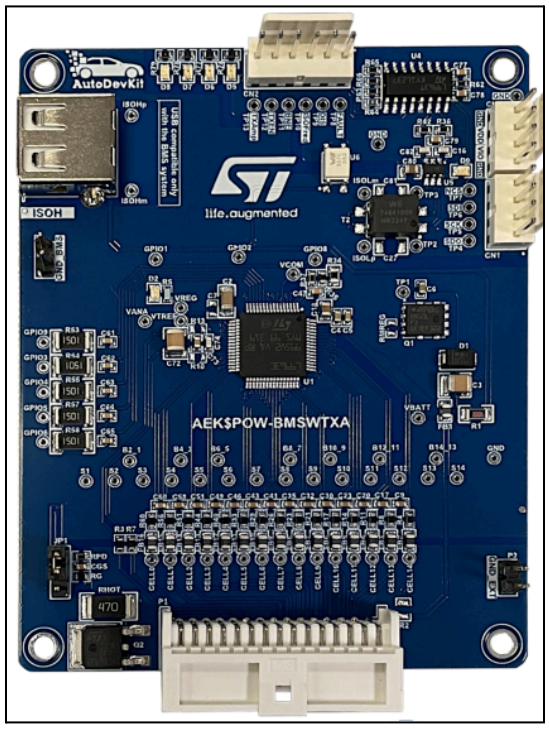


Figure 5: AEK-POW-BMSWTX evaluation board containing the L9963E AFE chip.

6. BATTERY MEASUREMENTS

The battery management system monitors individual cell-group voltages, pack current, and battery temperatures to determine the operating condition of the battery pack. These measurements are collected and processed by the STM32F411RE microcontroller and are used for battery monitoring, fault protection, and communication with the PC interface.

The firmware collects updated measurements during the main application loop. The processed values are then used to evaluate the battery's operating conditions and are included in the UART telemetry sent to the computer.

6.1 Cell Voltages

The battery pack consists of seven series-connected cell groups, with two lithium-ion cells connected in parallel within each group. The ST L9963E battery-monitoring IC measures the voltage of each series cell group and communicates the measurements to the STM32 microcontroller. The firmware reads the cell-voltage measurements from the battery-monitoring IC and converts them into voltage values. The total pack voltage is calculated by adding the voltages of the seven individual cell groups.

Individual cell-voltage monitoring is important because the total pack voltage does not indicate whether every cell group is operating within its allowed voltage range. For example, one cell group could reach an overvoltage condition while the overall pack voltage remains within its expected operating range. The firmware also uses the individual cell-voltage measurements to detect overvoltage and undervoltage conditions. The measurements are also transmitted to the PC interface, allowing the user to observe the voltage of each cell group during operation.

6.2 Pack Current

Pack current is measured using the system’s current-sensing circuitry through a power shunt resistor of 0.1 Ω. The resulting analog signal is read by the STM32’s analog-to-digital converter (ADC), and the firmware converts the ADC measurement into a current value in milliamperes.

The current measurement allows the system to monitor the amount of current flowing through the battery pack during the charge operation. It is also used by the firmware to evaluate the overcurrent protection condition. Lastly, it could have been used to determine proper SOC calculations, but timing and scheduling constraints prevented this behavior. The processed current value is included in the UART telemetry sent to the PC interface. This allows the user to observe changes in the pack current during charging and other operating conditions. The final firmware uses a current threshold of 2,000 mA for overcurrent protection. The implementation of this protection function is discussed further in Section 7.

6.3 Cell Temperatures

Battery temperature is monitored using three temperature-sensing inputs. The temperature sensors provide analog signals that are measured by the STM32's ADC. This is done by having a calibrated thermistor used in a voltage divider circuit. The thermistors are then placed on selected cells of the pack to measure those cells' temperature. A typical thermistor divider calculation is then performed to determine the final temperature value from the read voltage.

The firmware processes the ADC measurements and converts them into temperature values in degrees Fahrenheit. These measurements allow the system to monitor battery temperature and identify conditions that exceed the allowed operating limit.

The three temperature values are included in the UART telemetry and displayed through the PC interface. This allows the user to observe the monitored temperatures while the battery management system is operating.

The final firmware uses a temperature limit of 122 °F. If any of the three monitored temperatures reaches or exceeds this limit, the firmware disables the master relay through its protection logic. The implementation of this protection function is discussed further in Section 7.

7. FAULT PROTECTIONS

The battery management system includes fault-protection functions to prevent the battery pack from continuing to operate when monitored conditions exceed the established limits. The firmware evaluates individual cell-group voltages, pack current, and battery temperatures during the main application loop.

The STM32F411RE controls the master relay through a digital output named MASTER_SWITCH. The relay is enabled only when the monitored conditions satisfy the protection requirements. If a fault is detected, the firmware disables the relay-control output to disconnect the battery pack from the external circuit.

The protection thresholds implemented in the final firmware are shown in Table 1.

Table 1: Battery protection thresholds

Protection condition	**Threshold**
Cell undervoltage	Any Cell Below 2.75 V
Cell overvoltage	Any Cell Above 4.20 V
Pack overcurrent	2,000 mA or higher
Cell overtemperature	122 °F or higher

7.1 Over and Under Cell Voltage

The firmware monitors the voltage of each of the seven series-connected cell groups using measurements obtained from the L9963E battery-monitoring IC. These measurements are evaluated individually because an unsafe voltage condition can occur in one cell group even when the total pack voltage remains within its expected operating range.

An undervoltage fault is identified when an individual cell-group voltage falls below 2.75 V. An overvoltage fault is identified when an individual cell-group voltage exceeds 4.20 V. These values were selected as they are the recommended operating conditions on the used 18650 lithium ion cell’s data sheets.

The firmware also evaluates the validity of the voltage measurements. If the required voltage data is unavailable or invalid, the system does not treat the battery as being in a safe operating condition.

When an undervoltage fault, overvoltage fault, or invalid voltage measurement is detected, the firmware disables the master relay. This prevents the system from continuing

normal operation while the affected cell group is outside its allowed voltage range or its condition cannot be reliably determined.

7.2 Over Pack Current

The firmware monitors pack current using the measurement obtained from the current-sensing circuitry and processed by the STM32's ADC. The implemented overcurrent threshold is 2,000 mA. When the measured current reaches or exceeds this limit, the firmware disables the master relay through the MASTER_SWITCH output. This value was selected for safety purposes to prevent any damage to the system, despite the fact that much higher currents of 4 A or greater could be achieved. The current measurement is also included in the UART telemetry sent to the PC interface. This allows the user to observe the reported pack current during operation and monitor changes in current as the system operates. The current threshold provides an additional protection condition alongside the voltage and temperature limits. The battery pack also includes a physical 15 A fuse as a separate hardware protection component, to help if a software issue occurs or if the relay gets stuck.

7.3 Over Cell Temperature

The system monitors three battery-temperature measurements from thermistors to identify excessive temperature conditions. The STM32 processes the sensor measurements and converts them into temperature values in degrees Fahrenheit.

The implemented overtemperature threshold is 122 °F, equivalent to 50 °C. Each monitored temperature is evaluated individually. If any of the three measurements reaches or exceeds 122 °F, the firmware disables the master relay. This threshold was selected based on the recommended operating conditions of the used 18650 lithium ion cells from their data sheet. The temperature-protection function operates alongside the voltage and current protection functions. The master relay is enabled only when the monitored battery conditions satisfy all the implemented protection limits. Additionally, the temperature measurements are transmitted to the PC interface, allowing the user to observe the monitored battery temperatures during operation.

8. ACTIVE BALANCING

The topology chosen for our balancing circuit is called a switched capacitor balancing circuit. This type of circuit uses some form of switch, in our case, Solid State Relays (SSRs), to control which cells are connected to a capacitor. To balance the cells, first, the higher state of charge cell is connected, charging the capacitor up to its voltage level. Then, the higher SOC cell is disconnected, and a small dead-band waiting period occurs. Finally, the lower SOC cell is connected. The capacitor then discharges down to the same voltage level as the lower SOC cell. This process can be seen in Figure 6 which shows MOSFET switches instead of SSRs, but the concept is the same. After another dead-band waiting period, the cycle repeats until all cells are charged to the same SOC. The oscilloscope reading of this in work can be seen in Figure 7.

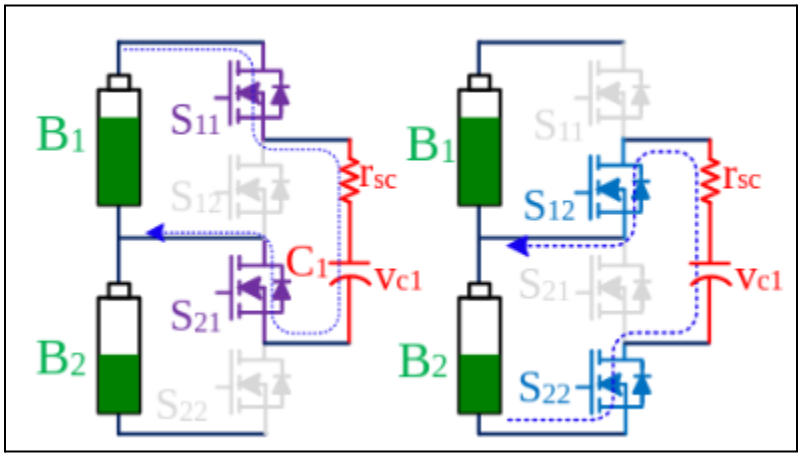


Figure 6: Balancing Topology and Operation for Two Cells

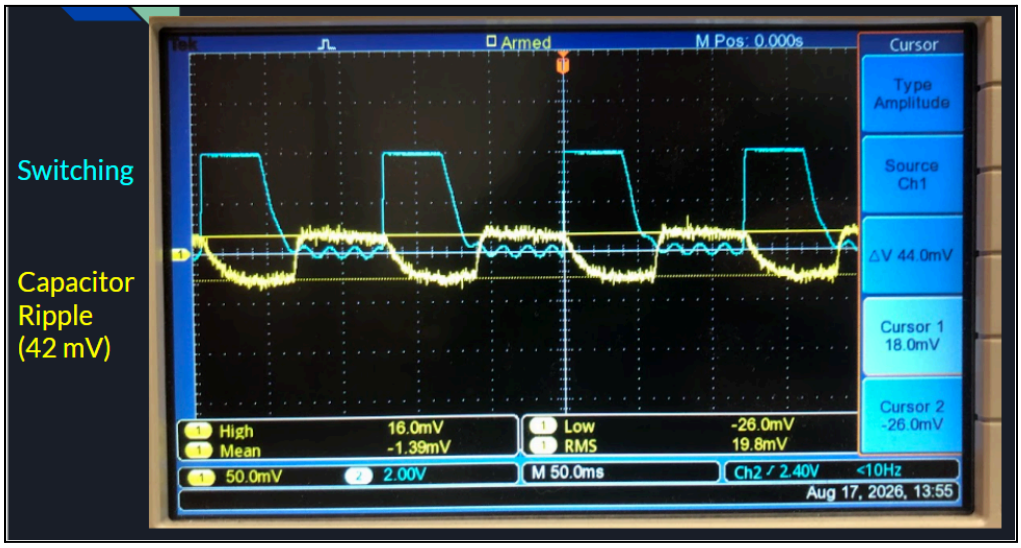


Figure 7: Oscilloscope Capture of Active Switching

This circuit was selected based on the simplicity of the controls involved and to avoid having to use the transformers common in other active balancing circuit topologies. This gave us more time to test and build for the final circuit, which helped us meet the strict timeline and budget requirements. In retrospect, while this circuit is simpler and cheaper, it likely moves charge from one cell to another much more slowly than a circuit that uses transformers, as the capacitors place a hard limit on the amount of amperage that can be used to charge them. This results in a minimal balancing current, meaning that a longer time period must pass in order for any significant cell balancing to take place.

9. DATA TELEMETRY AND PC INTERFACE

The battery management system includes a PC interface that allows the user to monitor battery measurements during operation. The STM32F411RE microcontroller transmits measurement data to the computer through UART communication, and a LabVIEW application receives and displays the information.

The firmware sends a formatted serial message containing the total pack voltage, individual cell-group voltages, battery temperatures, pack current, and voltage-fault information. This allows multiple battery measurements to be communicated through a single serial connection.

9.1 UART Data Telemetry

The STM32 uses its USART2 peripheral to transmit battery measurements to the computer at a baud rate of 115200. During the main application loop, the firmware collects the latest measurements and formats them into a text-based UART message. Each message includes the total pack voltage, the voltages of all seven series-connected cell groups, three temperature measurements, pack current, fault status, and voltage-measurement validity information. This message uses labels and separators to identify each measurement. This allows the PC application to distinguish the different values and display them in the appropriate locations. An example of the UART telemetry format is shown below:

PACK: 27.470 V | CELLS: 3.828 3.947 3.953 3.929 3.930 3.943 3.940 V | TEMP: 73 F 73 F 89 F | CURRENT: +7 mA | FAULT: OK | CELL: 0 | VALID[V:1]

The PACK field represents the calculated total pack voltage, while CELLS contains the seven individual cell-group voltages. The TEMP field contains the three monitored temperatures, and CURRENT reports the measured pack current. The FAULT and CELL fields report the voltage-fault status and associated cell information. The VALID field indicates whether the voltage measurements are considered valid.

The UART message is updated during the main application loop, allowing the PC interface to receive new measurements as the battery management system operates.

9.2 LabVIEW PC Interface

The PC interface was developed using LabVIEW 2025 to provide a visual display of the battery measurements received from the STM32. The application receives the serial data, parses the message to extract the individual measurements, and displays the values through numerical indicators and graphs. The interface includes graphs for individual cell voltages over time, cell voltage versus cell-group index, battery temperatures, and pack current.

The cell-voltage displays allow the user to observe differences between the seven series-connected cell groups. The temperature and current graphs provide additional information about the battery's operating conditions. All of these charts update in real time as the PC COM port receives new UART messages.

During system operation, the LabVIEW interface provides a convenient method of viewing the measurements without directly accessing the microcontroller or individual sensing circuits. It was also used during the final demonstration to show the battery's operating conditions while the active-balancing system was running and while fault conditions were being injected for master relay response. The completed LabVIEW interface is shown in Figure 8. It should be noted that the pack current waveform in the figure represents noise, as no load or charger was connected to the battery pack.

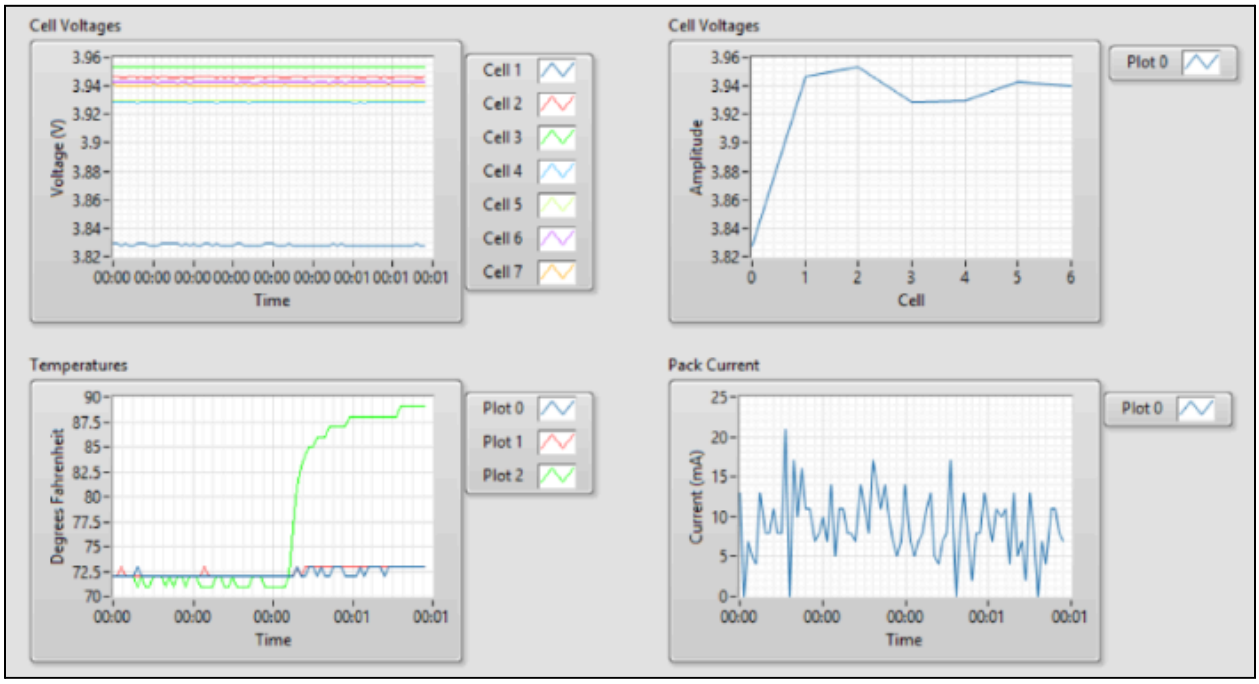


Figure 8. LabVIEW PC interface displaying cell voltages, battery temperatures, and pack current in real time.

10. SOLDERING AND BUILD

The circuit and components in this project were designed and selected to be simple, making assembly and soldering easier. The initial prototype was put together on a breadboard for testing and verification, as shown in Figure 9. After testing the circuit to verify the design would work, the final assembly could begin. Battery holders were modified to have the cells in parallel, which made it simple to connect cells together. This avoids the need to weld nickel strips to the batteries. The use of Wago nuts, JST connectors, and Anderson plugs made it simple to create connections by using off-the-shelf components rather than soldered connections. Because an evaluation board for the AFE chip was used, there was no need to solder a small chip to a board. Additionally, this board allowed connections between the AFE and MCU to be made with simple jumper wires, rather than a custom circuit board or a more complicated wiring harness. Finally, the balancing circuit was designed to use all through-hole components, enabling the use of a soldered breadboard and allowing all soldering to be completed much more quickly with far less risk of improper electrical connections. All of this resulted in the final circuit assembly seen in Figure 10, which was placed in the enclosure.

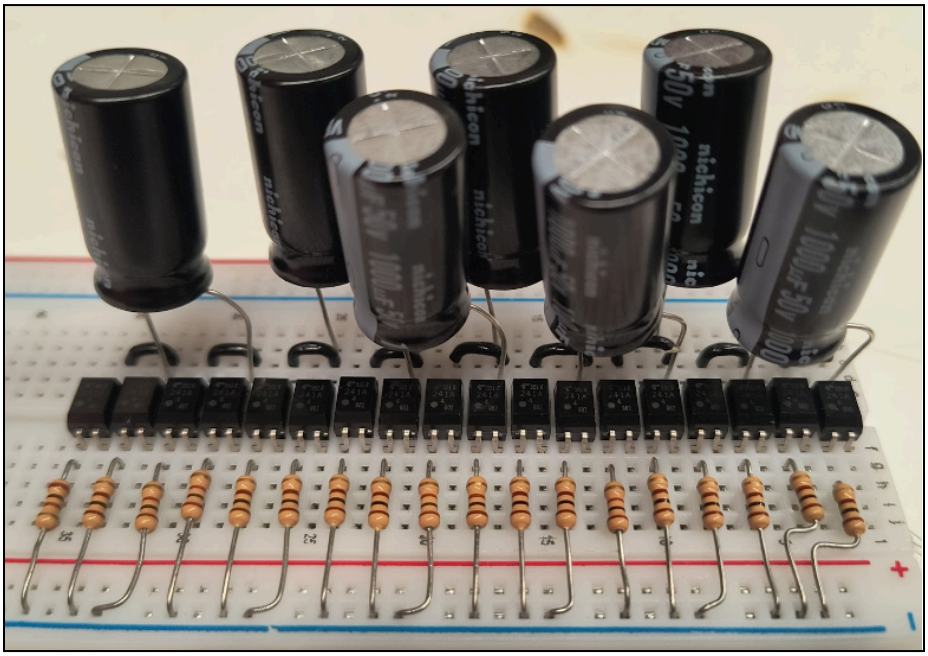


Figure 9. Breadboard Prototype of Active Balancing Circuit

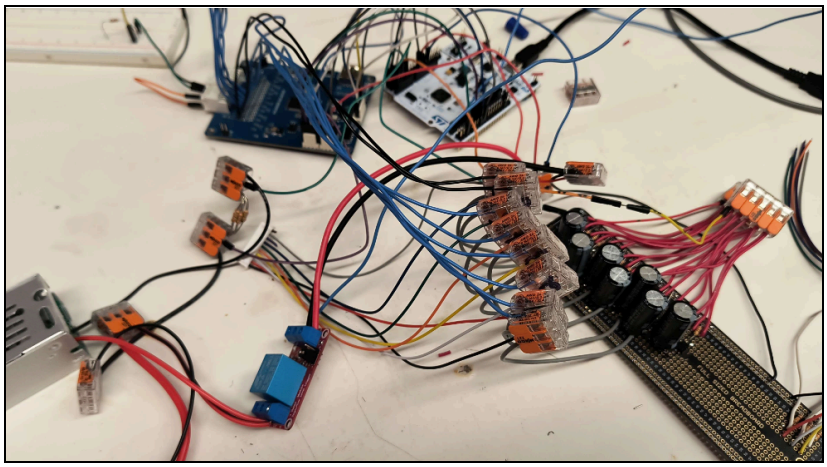


Figure 10. Final Wiring Circuitry with Solderable Protoboard and Wago Connectors being Utilized for Simplified Electrical Connections

11. ENCLOSURE

The design of the enclosure was done entirely inside SolidWorks, an industry-standard CAD software. This allows the team to use direct measurements, along with advanced tools and assembly methods, to virtually prove out everything was going to work. The first part of the process was making digital twins of all of the parts we were using. Some of these were able to be found on the manufacturer's website, but most of them were hand-measured and recreated in SolidWorks.

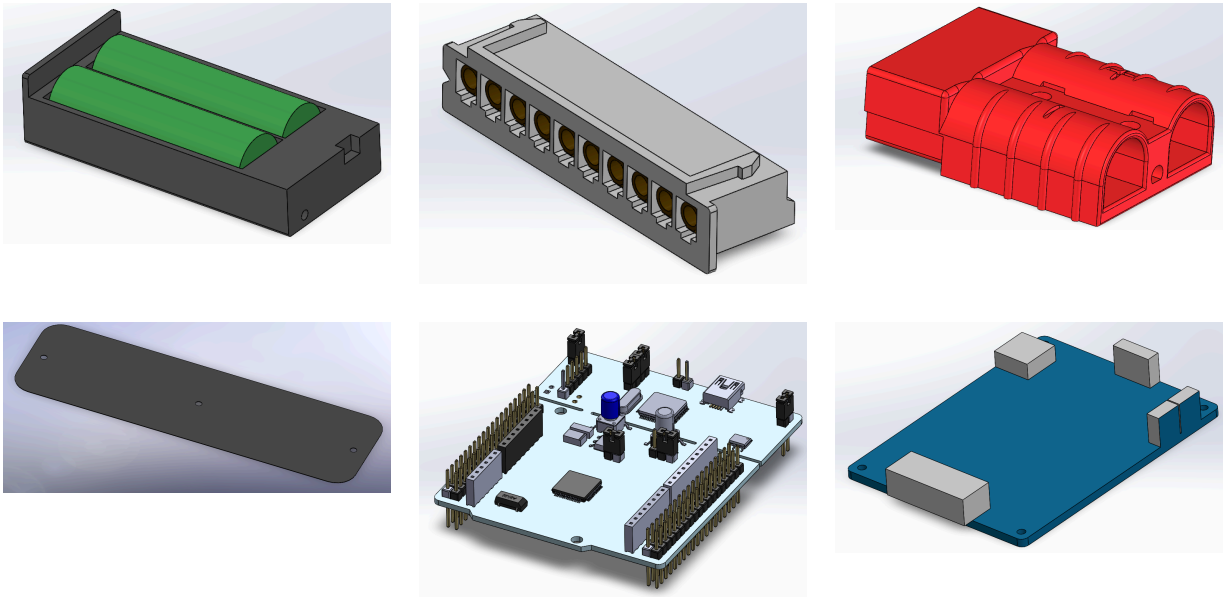


Figure 11. Digital twins of ordered parts

The enclosure needed to fulfill a few specific requirements of the project in order to be adequately designed. One of the major ones that was considered was the requirement that the battery pack itself be disconnectable from the control system. To facilitate this, Anderson connectors, along with JST connectors, were used to create a disconnectable link between the two containers. The Anderson connectors were used for the main voltage and current pull from the battery pack, while the JST connectors were used for the temperature sensing and individual cell sensing.

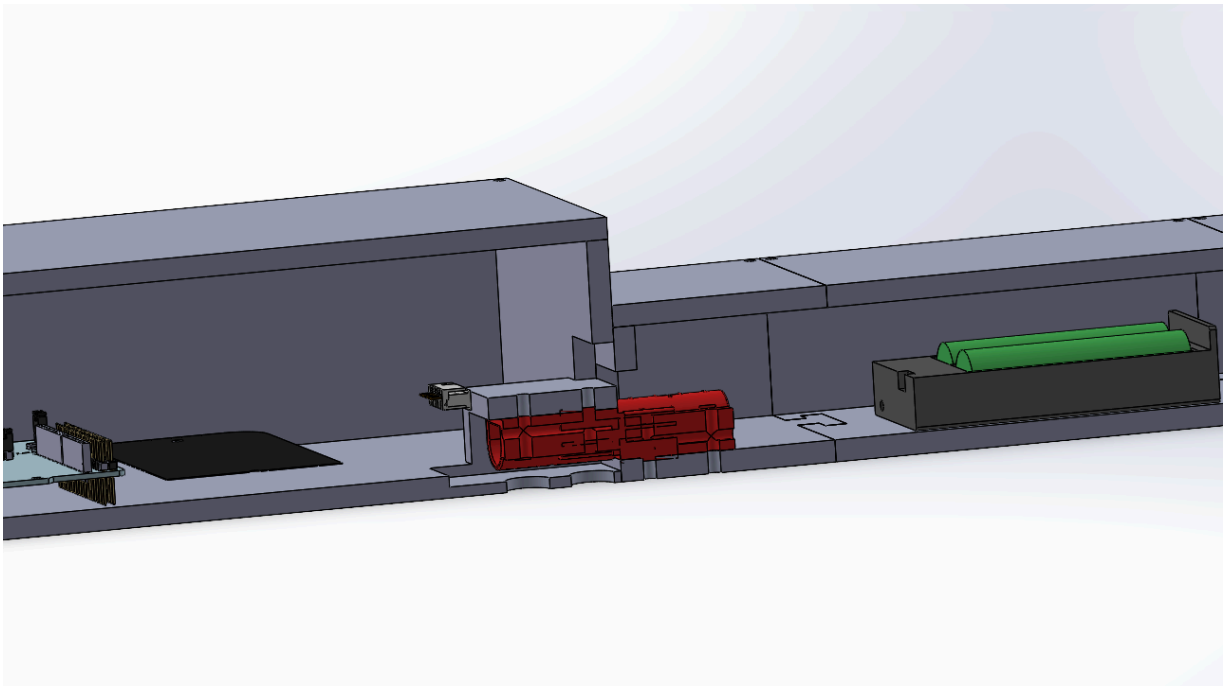


Figure 12. Cut-away view of battery and control connection

Another major issue was that, due to the way we are using the 18650 batteries being in a removable battery case instead of being spot-welded together, the size required for each of them was larger than it would otherwise be. The advantage of this, however, is that it makes it easy to replace an individual cell if need be due to battery health. To facilitate this, instead of tightly stacking all the batteries on top of each other, which would restrict access, they were laid out in a straight line, allowing direct access to the batteries. This introduces another issue, however, in that the space required to mount all of the batteries exceeds the maximum build plate size of the printer used. To fix this, the assembly was broken into submodules, with a fork-like connecting feature that slides them together after they are finished printing.

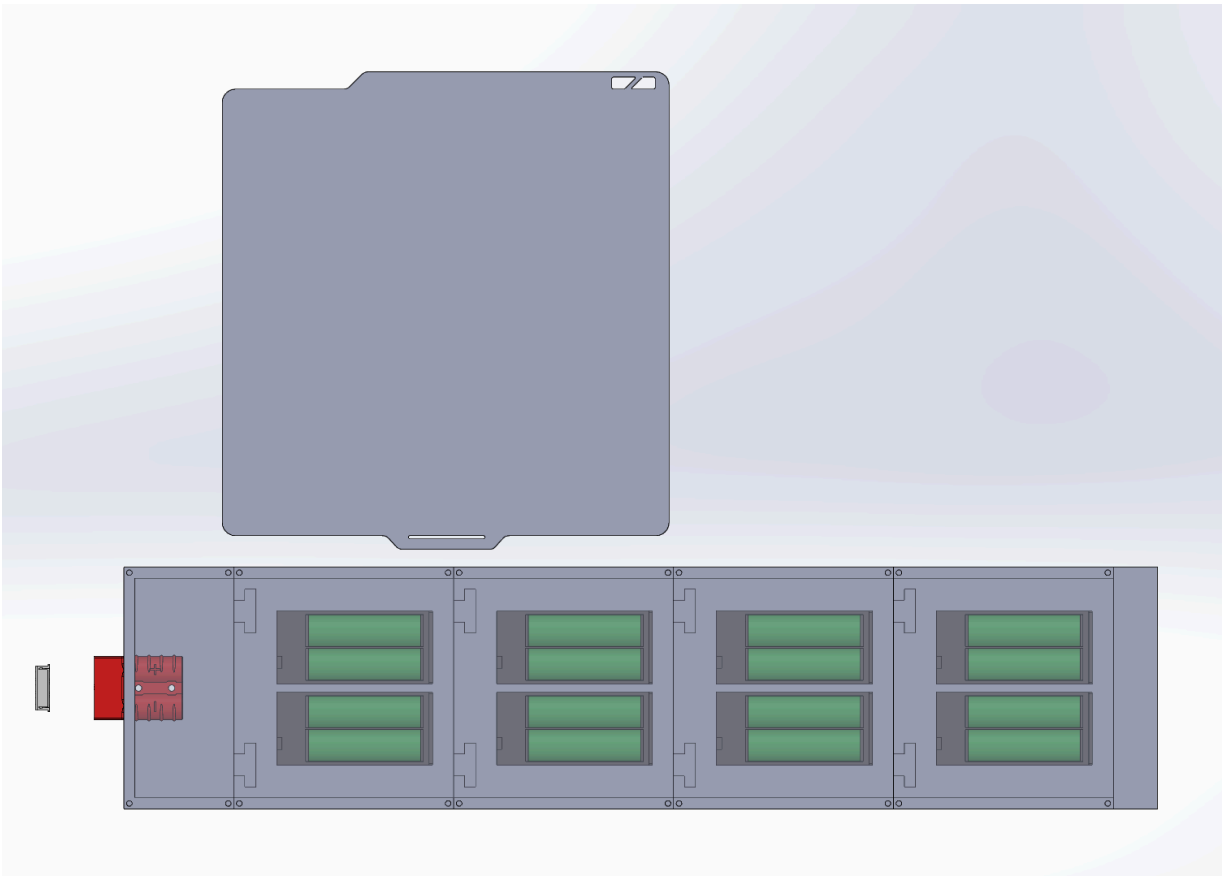


Figure 13. Build plate size vs final battery pack

Lastly, while there was not much special to report on for the control enclosure, there were still a few design iterations as the height and size needed to change as the final size of the controls came into perspective towards the end of the project.

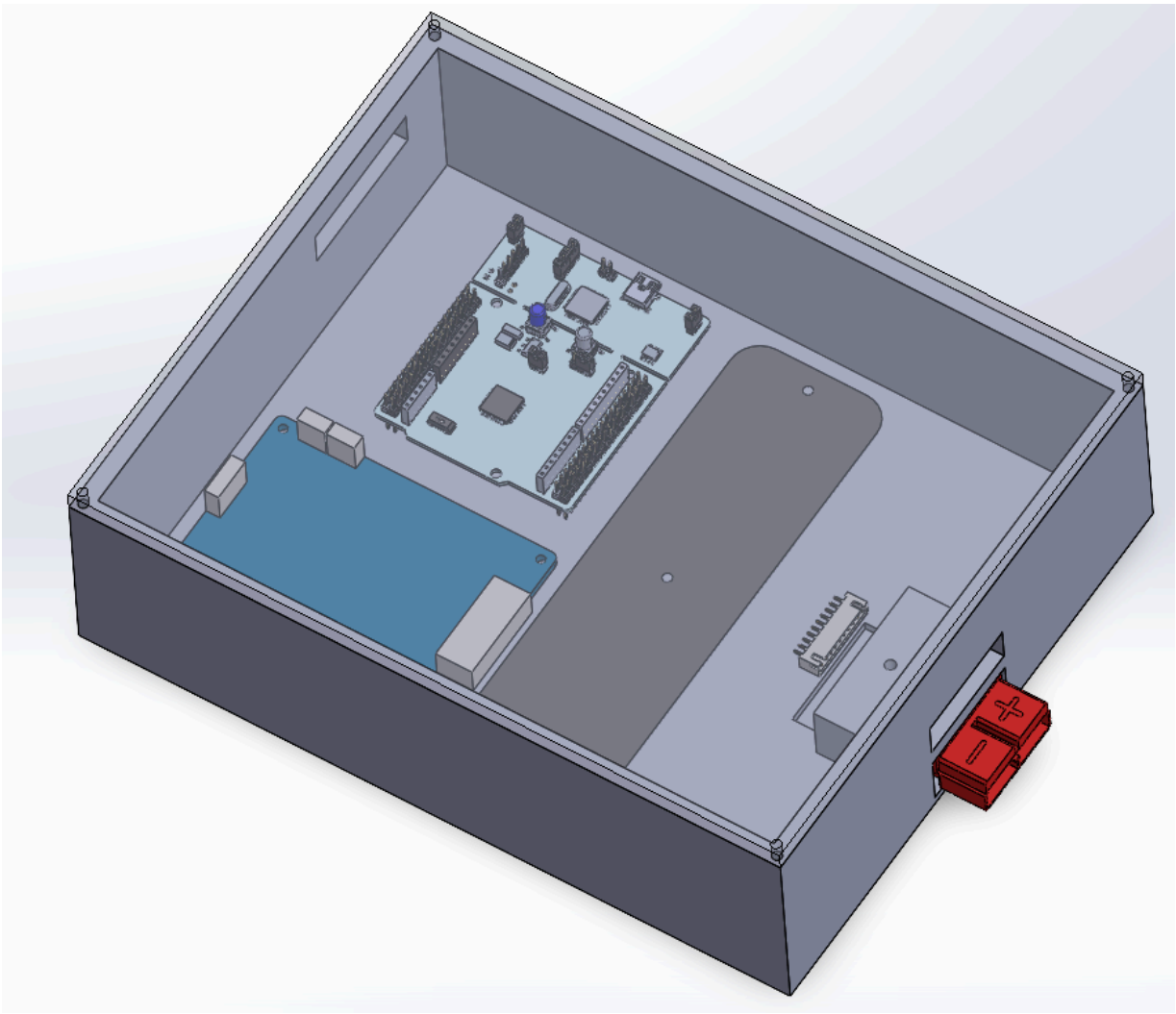


Figure 14. Control Enclosure

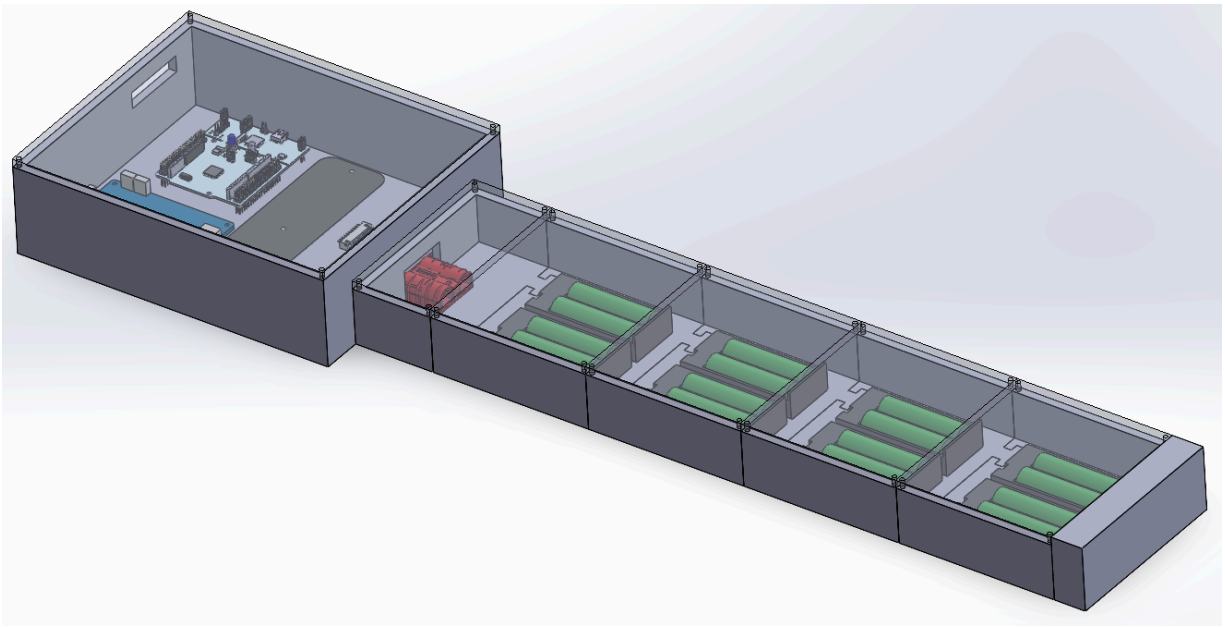


Figure 15. Final Enclosure Design

12. BILL OF MATERIALS AND BUDGET

The project was completed with a budget of $380.00. The team purchased the battery cells, battery-monitoring hardware, microcontroller development board, sensing components, switching devices, connectors, and other materials needed to construct the prototype.

Table 2 lists the components documented in the team's purchasing records used to prepare the project poster.

Table 2: Bill of Materials

Component	**Quantity**	**Unit Price**
Samsung 25R 18650 lithium-ion battery, 2500 mAh	18	$4.50
18650 two-cell battery holder	7	$1.49
18650/21700 battery charger	1	$12.99
Kapton tape, two-roll set	1	$7.99
18650 battery cell holder set, 100 pieces	1	$9.99
AEK-POW-BMSWTX battery-monitoring module	1	$73.28
10 kΩ NTC thermistor	7	$1.518
JST-XH 10-pin connector kit	1	$8.99
NUCLEO-F411RE development board	1	$18.82
TLP241A solid-state relay	20	$1.574
10 A fuse	1	$3.97
Solderable breadboard	1	$9.89
WAGO 221-413 connector	1	$25.35
50 A Anderson-compatible connector	1	$9.99
10 A relay	1	$6.99
Adjustable DC-DC buck-boost converter	1	$8.90

The battery cells and battery-monitoring module were essential components of the system. The STM32 development board provided the main controller, while the thermistors,

relays, fuse, and supporting circuitry were used for sensing, protection, and switching. The remaining materials supported the electrical connections, battery-pack assembly, and integration of the prototype.

The final budget figures reported by the team are shown in Table 3.

Table 3: Project budget summary

Budget item	**Amount**
Total project budget	$380.00
Total reported spending (including shipping)	$369.07
Remaining budget	$10.93

The team used approximately 97.1% of the available project budget, leaving $10.93 remaining. The completed prototype was therefore developed within the allocated budget.

13. PROJECT MANAGEMENT

The project was completed by a four-member team consisting of Michael Collins, Matthew Herber, Dillon Peterson, and Ryan Rabideau. The team divided the project into several areas, including battery-pack construction, battery-monitoring hardware, active balancing, embedded firmware, PC communication, and mechanical design. Each member took responsibility for different parts of the system while working together during integration and testing.

Throughout the project, the team held regular meetings to discuss progress, identify problems, and determine the next steps. These meetings helped coordinate the development of the individual subsystems and ensured that the electrical, software, and mechanical components could be integrated into the final prototype.

GitHub was used to manage the embedded firmware and track software changes. Team members worked on their respective development branches and integrated completed changes into the main project. This allowed the firmware to be developed and updated as the hardware became available and new measurement and control functions were added.

The project was developed in stages. Initial work focused on selecting components, establishing communication between the microcontroller and battery-monitoring hardware, and designing the battery pack and supporting circuitry. Later work included integrating the voltage, current, and temperature measurements, implementing protection functions, developing the active-balancing system, and establishing communication with the LabVIEW PC interface.

As the final demonstration approached, the team focused on combining the individual subsystems, troubleshooting hardware and software issues, and preparing the completed prototype. The team also worked together to prepare the project poster, presentation, and final report.

Communication and coordination were important throughout the project because several parts of the system depended on one another. For example, firmware development required access to the completed sensing and switching hardware, while system testing required the hardware, firmware, and PC interface to operate together. Dividing responsibilities while maintaining regular communication allowed the team to address these dependencies and complete the integrated prototype.

14. WORKLOAD DISTRIBUTION

14.1 Michael Collins - Initial Design and Procurement, Electrical/Hardware Design, Balancing Circuit Design, Microcontroller Firmware Development, AFE Interaction and Communication, PC Interface Development, Integration Testing

Michael was involved in many aspects of the project. He played a heavy part in researching key components in the initial design, such as the microcontroller and AFE boards to use. He was responsible for all electrical connections between the battery pack, active balancing circuit, AFE, microcontroller, and other components such as the master relay and shunt resistor. Additionally, he helped determine the optimal components to use in the balancing circuit design in order to try to maximize the balancing current. He was solely responsible for interaction with the AFE evaluation board and getting it to communicate with the microcontroller. He configured all the pins of the microcontroller, as well as partially designed and developed the software superloop used. He quickly developed a LabVIEW COM to graph PC interface for the data telemetry to be displayed in a lively manner. Lastly, he spent many late-night hours testing and debugging the final fixture in the design lab in order to ensure its proper functionality by the required deadline.

14.2 Matthew Herber - Enclosure Design & Production

Matthew created and produced the enclosure used for the project, creating multiple revisions and managing multiple 3D printers to get the final product. All work was done through SolidWorks, and pushed to the GIT repository as changes were made, with multiple design iterations to do fitment testing and redesigns when parts had problems.

14.3 Dillon Peterson - Research, Balancing Circuit Design, Testing, Wiring, and Soldering

Dillon completed the initial topology research for the project, helping the team identify the simpler switched-capacitor balancing circuit and finding the SSRs that were eventually selected. With that research, he designed and prototyped a balancing circuit to be made on a breadboard. After assisting with testing the prototype, he completed the final assembly, including soldering the balancing circuit, wiring the batteries to the required connections, and assisting in connecting the MCU to the balancing circuit. Additionally, he helped to prototype the thermistor circuit and programming, helped to order components, and built a simple Simulink model of the switching topology. Finally, Dillon put the poster for the final demo together and helped in reports and updates as needed.

14.4 Ryan Rabideau - Firmware Integration, Battery Monitoring, and Protection

Ryan contributed to the development and integration of the embedded firmware for the battery management system. His work included processing cell-voltage measurements from the

L9963E battery-monitoring IC, integrating temperature and pack-current measurements into the main application, and implementing battery fault-detection and protection functions.

He also developed the UART telemetry functionality used to transmit battery measurements and system-status information from the STM32 microcontroller to the PC interface. This involved formatting the cell voltages, total pack voltage, temperatures, current, and fault information into a serial message that could be received and displayed by the LabVIEW application.

Ryan assisted with firmware integration and troubleshooting as the individual hardware and software components were combined into the final system. He also contributed to the project poster, technical documentation, and final report.

14.5 Overall Workload Estimate

In total, it took about an estimated 150 man-hours in about 8 weeks in order to complete this project. Many hours were spent outside of lecture and lab in order to develop and prototype the described design. These hours did not include creating client meeting presentations, the final posterboard presentation, or this final report.

15. FINAL PROJECT OUTCOMES

The team completed the development and integration of a battery management system for a nominal 24 V lithium-ion battery pack. The final prototype combines a 2P7S battery pack, battery-monitoring hardware, an STM32F411RE microcontroller, active-balancing circuitry, temperature and current sensing, electrical protection, and a LabVIEW PC interface.

The firmware was integrated with the battery-monitoring hardware to collect measurements from the seven series-connected cell groups. The system also measured pack current and three battery temperatures. These measurements were processed by the microcontroller and transmitted to the PC interface, allowing the user to observe the battery's operating conditions.

The completed firmware includes protection logic for cell overvoltage, cell undervoltage, pack overcurrent, and battery overtemperature. The master relay is controlled based on these conditions so that the battery pack can be disconnected when the monitored values exceed the implemented limits.

The active-balancing circuitry was integrated with the battery pack and microcontroller. During the final project demonstration, the team demonstrated active balancing during charging while displaying battery measurements through the LabVIEW interface.

The project also resulted in a physical prototype containing the battery pack, electronics, and supporting hardware. The team completed the system assembly and prepared the prototype for the final demonstration and presentation.

The prototyped BMS system was able to fulfill almost all of the project requirements: (1) the master relay control, in-line fuse, and isolated electrical connections provided safety; (2) the separated and spacious enclosures provided layout organization; (3) the battery pack and BMS each had their own dedicated enclosures; (4) the BMS was able to monitor all cell voltages, pack voltage, three cell temperatures, and the battery pack current while charging; (5) a switching capacitor ladder active balancing topology was implemented (granted that the balancing current was rather small); (6) overvoltage, undervoltage, and overtemperature protection were implemented by having the microcontroller scanning for these faults and controlling the master relay accordingly; (7) the BMS could potentially calculate the battery pack state of charge with ease through the measured pack current when charging, however, this trivial requirement was not fully implemented due to lack of time; lastly, (8) the BMS outputs all telemetry data to a PC interface which could be recorded for future analysis and reference. Additionally, the bonus requirement of overcurrent protection was fulfilled with both the actively controlled relay based on fault conditions and the replaceable fuse being placed in line with the battery pack terminals.

This project has taught our group invaluable lessons in power circuits, microcomputer system design, and of course, battery management and balancing. We attribute the majority of the project's success to our simplified design which only used off the shelf components and evaluation boards to reduce fabrication, soldering, and build time. Overall, the project brought together the individual electrical, software, and mechanical subsystems into a working battery management system prototype. The final demonstration showed the integration of battery monitoring, PC communication, and active balancing.

16. CONCLUSIONS

The objective of this senior design project was to develop a battery management system for a nominal 24 V lithium-ion battery pack. The team completed a prototype that combines a 2P7S battery pack, an STM32F411RE microcontroller, an ST L9963E battery-monitoring IC, active-balancing circuitry, battery protection, and a LabVIEW PC interface.

The completed system monitors individual cell-group voltages, pack current, and battery temperatures. The firmware processes these measurements, evaluates the implemented protection conditions, controls the master relay, and transmits battery information to the PC interface. During the final demonstration, the team demonstrated active balancing during charging while displaying battery measurements through LabVIEW.

Throughout the project, the team gained experience with battery-pack construction, circuit design, embedded firmware development, mechanical design, and hardware and software integration. Combining the individual subsystems required troubleshooting, testing, and communication between team members to resolve issues and complete the final prototype.

Although the project resulted in a working prototype, several improvements could be made in future development. These include additional testing of the protection functions under controlled conditions, further calibration of the sensing circuitry, and completing and validating state-of-charge estimation and display functionality.

Overall, the project provided valuable experience in designing and integrating a complete engineering system. The team successfully combined the electrical, software, and mechanical components into a battery management system and demonstrated its operation during the final project presentation.